4.Write a Java program to demonstrate a) packages

Type	Input	Expected Output	Explanation
Success	Import user-defined package	Package class method executed	Package usage
Success	Use built-in java.util.Scanner package	Input read successfully	Built-in package
Success	Multiple classes in package	Correct class accessed	Package organization
Failure/Edge	Missing import statement	Compilation error	Dependency error
Failure/Edge	Wrong package name	Class not found error	Package path issue

Algorithm: PackageDemo

1. Start.
2. Import the user-defined package classes:
    - `Message`
    - `Helper`
3. Import the built-in `Scanner` class from `java.util`.
4. Create an object of the `Message` class.
5. Call the `showMessage()` method to display the message.
6. Create an object of the `Helper` class.
7. Call the `showHelper()` method to display the helper message.
8. Create a `Scanner` object to read user input.
9. Prompt the user to enter their name.
10. Read the name entered by the user using `nextLine()`.
11. Display the entered name.
12. Close the `Scanner` object to release resources.
13. Stop.

Program:

Message.java

```
package mypackage;
public class Message {
 public String showMessage() {
 System.out.println(" Package class method executed");
}
}
```

Helper.java

```
package mypackage;
public class Helper {
 public String  showHelper() {
 System.out.println(" Helper class accessed successfully");
}
}
```

PackageDemo.java

```
import  mypackage.Message;
import  mypackage.Helper;
```

```
import  java.util.Scanner;

public class PackageDemo {
   public static void main(String[] args) {

       // User-defined package usage
       Message msg = new Message();
       msg.showMessage();

       // Multiple classes in package
       Helper helper = new Helper();
       helper.showHelper();

       // Built-in package usage
       Scanner sc = new Scanner(System.in);

       System.out.print("Enter your name: ");
       String name = sc.nextLine();

       System.out.println("Input read successfully: " + name);

       sc.close();
   }
}
```

b) abstract classes and interfaces.

Type	Input	Expected Output	Explanation
Success	Circle extends Shape	Area calculated	Abstract method implementation
Success	Class implements Printable	Print method executed	Interface implementation
Success	Abstract class with interface	Both methods executed	Combined use
Failure/Edge	Instantiate abstract class	Compilation error	Abstract class cannot be instantiated
Failure/Edge	Interface method not overridden	Compilation error	Interface contract

Algorithm:

1. Start.
2. Define an interface `Printable` with the method `print()`.
3. Define an abstract class `Shape`:
    - Declare an abstract method `calculateArea()`.
    - Define a concrete method `showShape()`.
4. Create class `Circle` that extends `Shape`:
    - Store the radius.
    - Implement `calculateArea()` using the formula:
        - `Area = π × radius × radius`.
5. Create class `Document` that implements `Printable`:
    - Implement the `print()` method to display a message.
6. Create class `Rectangle` that extends `Shape` and implements `Printable`:
    - Store the length and breadth.
    - Implement `calculateArea()` using the formula:
        - `Area = length × breadth`.
    - Implement the `print()` method to display rectangle details.

7. In the `main()` method:
    - Create a `Circle` object with radius `5`.
    - Calculate and display the circle's area.
    - Create a `Document` object.
    - Call its `print()` method.
    - Create a `Rectangle` object with length `10` and breadth `5`.
    - Calculate and display the rectangle's area.
    - Call its `print()` method.
8. Note the edge cases:
    - An abstract class (`Shape`) cannot be instantiated directly.
    - Any class implementing `Printable` must override the `print()` method; otherwise, a compilation error occurs.
9. Stop.

Program:

```
// Interface
interface Printable {
    void print();
}

// Abstract Class
abstract class Shape {
    abstract double calculateArea();

    void showShape() {
        System.out.println("Shape method executed");
    }
}

// Circle extends abstract class
class Circle extends Shape {
    private double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    //@Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}

// Class implementing interface
class Document implements Printable {

    //@Override
    public void print() {
        System.out.println("Print method executed");
    }
}
```

```
// Abstract class + Interface combined
class Rectangle extends Shape implements Printable {
    private double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }

   // @Override
    double calculateArea() {
        return length * breadth;
    }

    //@Override
    public void print() {
        System.out.println("Rectangle details printed");
    }
}

public class AbstractInterfaceDemo {
    public static void main(String[] args) {

        // Test Case 1: Circle extends Shape
        Circle circle = new Circle(5);
        System.out.println("Circle Area = " + circle.calculateArea());

        // Test Case 2: Interface implementation
        Document doc = new Document();
        doc.print();

        // Test Case 3: Abstract class with interface
        Rectangle rect = new Rectangle(10, 5);
        System.out.println("Rectangle Area = " + rect.calculateArea());
        rect.print();

        /*
         * Failure / Edge Cases
         */

        // 1. Cannot instantiate abstract class
        /*
        Shape s = new Shape(); // Compilation Error
        */

        // 2. Interface method not overridden
        /*
        class Test implements Printable {
            // Missing print() method
        }
        // Compilation Error
        */
    }
}
```